

**Escuela Politécnica Nacional
Facultad De Ingeniería De Sistemas
Ingeniería De Software
Construcción Y Evolución De Software
(ISWD633)
GR2SW**



**Proyecto: Documento de Construcción y
Evolución de Software**

Umami - Sistema de Recetas

Chicaiza Andrea

31/01/2026

1. Introducción

El proyecto Umami es un sistema de búsqueda de recetas culinarias desarrollado con la finalidad de facilitar al usuario la planificación de comidas a partir de los ingredientes que tiene disponibles en su hogar. El sistema resuelve un problema cotidiano: aprovechar ingredientes existentes sin necesidad de realizar compras adicionales, reduciendo así el desperdicio de alimentos y fomentando la creatividad gastronómica.

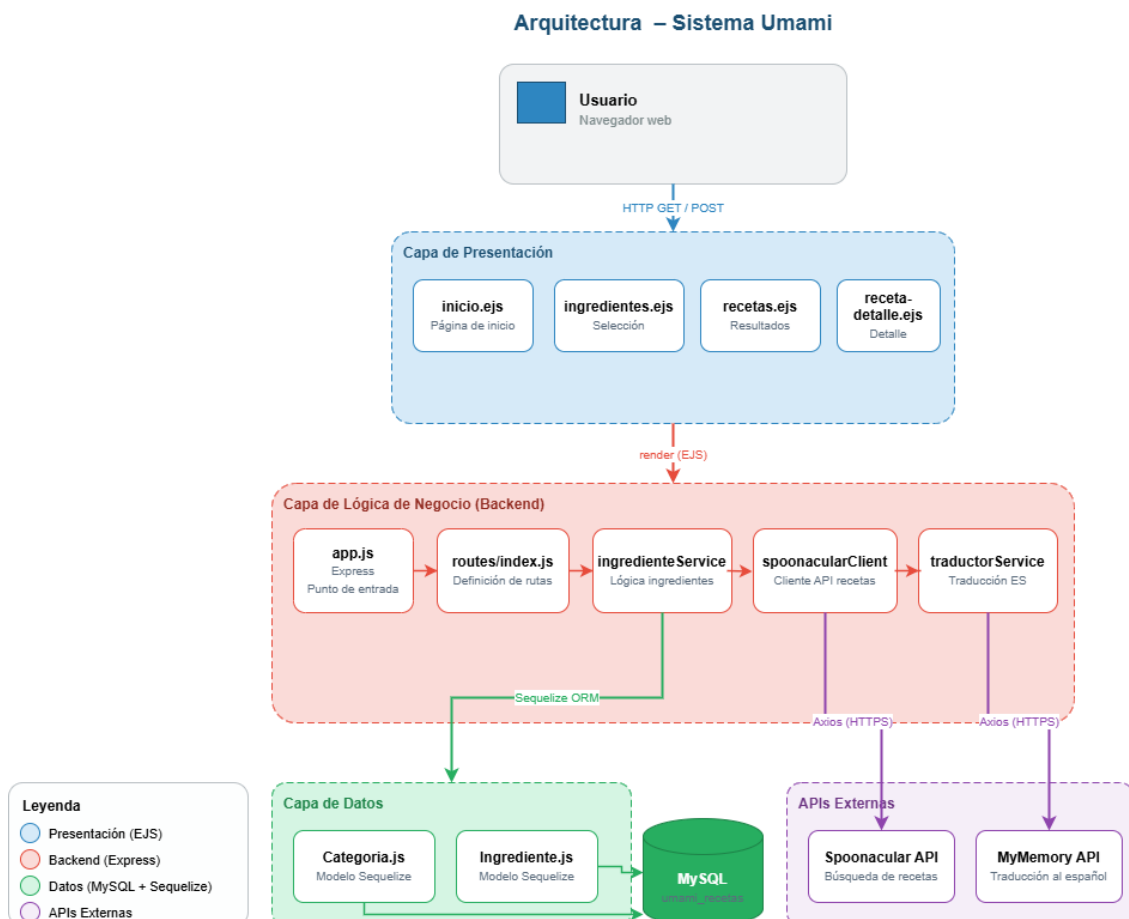
Para lograr este objetivo, la aplicación integra la API externa de Spoonacular como fuente principal de datos de recetas, y utiliza la API de MyMemory para traducir automáticamente los resultados al idioma español. De esta forma, el usuario puede interactuar con el sistema de manera sencilla y obtener resultados relevantes en tiempo real.

El presente documento tiene por objeto describir de manera detallada la arquitectura del sistema, las estrategias de integración continua y entrega continua (CI/CD), el modelo de flujo de desarrollo adoptado, y las prácticas de revisión y control de calidad implementadas durante la construcción del software. Su propósito es demostrar cómo se ha gestionado la evolución del proyecto desde su concepción hasta su estado actual, garantizando trazabilidad, calidad y sostenibilidad del código.

2. Arquitectura del Proyecto

La arquitectura de Umami se estructura bajo un modelo cliente-servidor tradicional, donde el servidor actúa como punto central de orquestación entre la interfaz de usuario, la base de datos y las APIs externas. A continuación, se describe cada componente principal y la forma en que estos se relacionan entre sí.

2.1. Diagrama de alto nivel



2.2. Descripción de componentes

El backend del proyecto está desarrollado en Node.js utilizando el framework Express, que permite definir rutas HTTP de forma clara y modular. Esta capa se encarga de recibir las solicitudes del navegador, ejecutar la lógica de negocio correspondiente y devolver las respuestas renderizadas mediante el motor de plantillas EJS.

La base de datos utilizada es MySQL, gestionada mediante el ORM Sequelize. Este permitió definir los modelos de datos —categorías e ingredientes— de forma programática, facilitando las operaciones de lectura y escritura sin necesidad de escribir consultas SQL directamente en el código de la aplicación.

Para la obtención de recetas, el sistema se comunica con la API de Spoonacular a través del cliente HTTP Axios. El módulo spoonacularClient.js encapsula toda la lógica de comunicación con dicha API externa, manteniendo el código principal desacoplado de los detalles de la integración. De manera análoga, el servicio de traducción utiliza la API gratuita de MyMemory para convertir los nombres y descripciones de las recetas al español antes de presentarlos al usuario.

2.3. Estrategia de integración entre módulos

La integración de los distintos módulos sigue un patrón de capas claras: las rutas definidas en routes/index.js delegan la responsabilidad de la lógica de negocio a los servicios ubicados en la carpeta services/. Estos servicios, a su vez, interactúan con los modelos de Sequelize o con las APIs externas según corresponda. Esta separación permite que cada componente pueda ser probado y modificado de forma independiente.

Componente	Tecnología	Responsabilidad
Frontend (vistas)	EJS	Renderizado de plantillas HTML
Backend (servidor)	Node.js + Express	Lógica de negocio y rutas
Base de datos	MySQL + Sequelize	Almacenamiento de categorías e ingredientes
API recetas	Spoonacular (Axios)	Búsqueda de recetas según ingredientes
API traducción	MyMemory (Axios)	Traducción automática al español

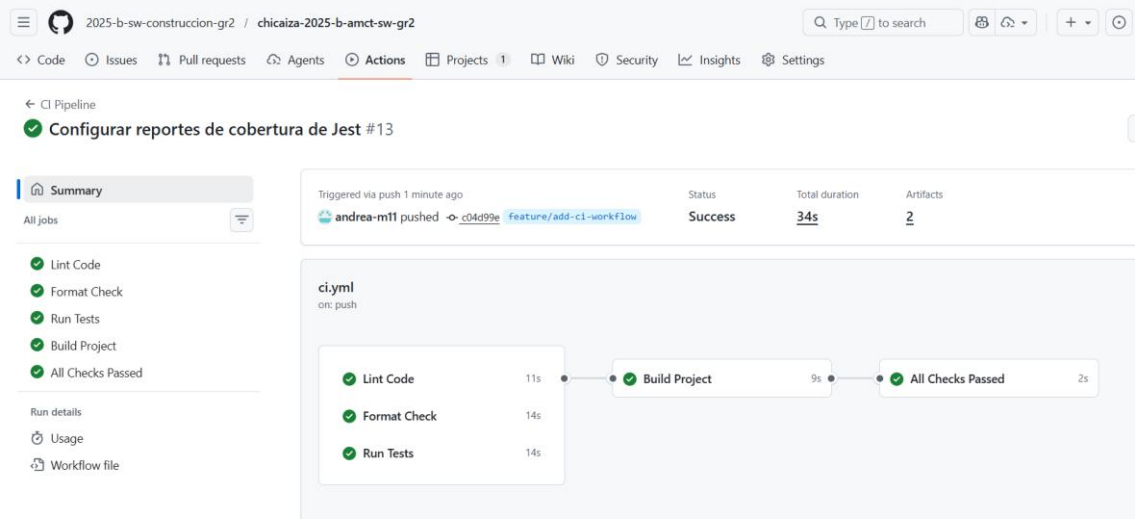
3. Estrategia de Pipelines (CI/CD)

El proyecto implementa un pipeline de Integración Continua (CI) mediante GitHub Actions, definido en el archivo .github/workflows/ci.yml. Este pipeline se ejecuta automáticamente cada vez que se realiza un push o se abre un Pull Request hacia las ramas main, develop o cualquier rama que siga el patrón feature/**. Su objetivo es verificar de forma temprana que el código nuevo no introduzca errores ni reduzca la calidad del proyecto.

3.1. Pipeline de Integración Continua

El pipeline se compone de varios jobs que se ejecutan en un orden específico. Primero, de manera en paralelo, se ejecutan tres tareas independientes: el análisis estático del código con ESLint, la verificación del formato con Prettier, y la ejecución de las pruebas unitarias con Jest. Estas tres tareas no dependen entre sí, por lo que pueden correr simultáneamente, reduciendo el tiempo total de validación.

Una vez que los tres jobs anteriores han finalizado exitosamente, se ejecuta el job de Build, que compila el proyecto y genera los artefactos de distribución en la carpeta /dist. Finalmente, un job de Status Check actúa como verificación final: solo si todos los pasos previos han pasado, el pipeline se considera exitoso. Este mecanismo impide que código que no cumple con los estándares del proyecto sea fusionado en ramas principales.



3.2. Detalle de cada job

Job	Herramienta	Descripción
Lint	ESLint	Análisis estático para detectar errores y malas prácticas de codificación.
Format	Prettier	Verificación de que el código sigue el estilo definido en la configuración del proyecto.
Tests	Jest	Ejecución de pruebas unitarias y generación de reporte de cobertura de código.
Build	npm run build	Compilación del proyecto y generación de artefactos listos para despliegue.
Status Check	GitHub Actions	Verificación final que confirma que todos los jobs anteriores fueron exitosos.

3.3. Características adicionales del pipeline

El pipeline incluye mecanismos de optimización y reportado. Se utiliza caché de dependencias de npm para acelerar las instalaciones en ejecuciones futuras. Los reportes de cobertura de código se suben automáticamente a Codecov, permitiendo un seguimiento visual del porcentaje de código cubierto por pruebas. Además, los artefactos de cobertura y de build se conservan durante 14 días para permitir revisiones posteriores. Por último, el sistema de concurrencia configurado cancela ejecuciones anteriores del pipeline si se detecta un nuevo push en la misma rama, evitando el desperdicio de recursos.

3.4. Pipeline de Entrega Continua

Además de la integración continua, el proyecto implementa una estrategia de Entrega Continua (CD) que extiende el pipeline de CI hacia el despliegue automático del artefacto generado. Una vez que el job de Build ha finalizado exitosamente y el Status Check confirma que todos los pasos

previos han sido aprobados, el artefacto compilado queda listo para ser desplegado en un entorno de staging.

El despliegue al entorno de staging se ejecuta de manera automática como paso inmediatamente posterior al Build, siempre que la rama origen sea develop. Este entorno permite realizar pruebas de integración y verificaciones manuales en condiciones similares a las de producción, sin afectar el entorno activo. Los artefactos desplegados en staging se conservan durante 14 días, coincidiendo con la política de retención de artefactos del pipeline.

El despliegue al entorno de producción, asociado a la rama main, no se ejecuta de forma automática. En cambio, requiere la aprobación explícita mediante un Pull Request que haya sido revisado y aprobado por al menos un miembro del equipo. Solo tras esta aprobación, y una vez que el pipeline CI ha validado los cambios, se permite la fusión hacia main y se activa el despliegue en producción. Este mecanismo de aprobación manual en la etapa final actúa como control adicional, garantizando que únicamente código revisado y validado llegue al entorno productivo.

Etap	Entorno destino	Condición de activación
Deploy automático	Staging	El pipeline CI finaliza exitosamente en la rama develop.
Deploy con aprobación	Producción	PR aprobado por al menos un revisor y pipeline CI exitoso en la rama main.

Esta estructura de dos niveles de despliegue —automático hacia staging y controlado hacia producción— es coherente con las mejores prácticas de entrega continua, ya que permite mantener un entorno de pruebas siempre actualizado mientras se preserva la estabilidad del entorno en producción.

4. Estrategia de Flujos de Desarrollo

El equipo adoptó una estrategia de ramas basada en GitFlow, un modelo ampliamente utilizado en proyectos de desarrollo colaborativo. Esta estrategia permite organizar el trabajo de forma clara, separando el código en producción de aquel que está en desarrollo o en proceso de revisión.

4.1. Modelo de ramas

Rama	Propósito
main	Contiene el código en producción. Es la versión estable del proyecto y no se modifica directamente.
develop	Rama de integración donde se fusionan las nuevas funcionalidades una vez aprobadas. Representa el estado actual del proyecto en desarrollo.
feature/*	Ramas creadas para el desarrollo de funcionalidades específicas. Cada una se nombra de forma descriptiva y se fusiona hacia develop.

4.2. Flujo de trabajo

Cuando un miembro del equipo necesita desarrollar una nueva funcionalidad, crea una rama feature/* a partir de develop. Una vez completado el desarrollo, se abre un Pull Request hacia

develop. Este PR debe incluir una descripción clara de los cambios realizados y debe ser revisado y aprobado por al menos un compañero del equipo antes de ser fusionado. Además, el pipeline de CI debe haber finalizado exitosamente como requisito previo al merge.

En el transcurso del proyecto se creó, entre otras, la rama feature/add-ci-workflow, dedicada a la configuración del pipeline de CI y a la integración de las herramientas de linting y testing. Esta rama fue fusionada a develop tras cumplir con todos los requisitos de revisión y validación.

Branches

New branch

OverviewYoursActiveStaleAll

Q Search branches...

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	37 minutes ago	5 / 5			Default

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
develop	44 minutes ago	10 / 10	1	0	#4
feature/add-ci-workflow	53 minutes ago	10 / 10	3	0	#5

5. Gestión de Historias de Usuario

Las historias de usuario son una herramienta fundamental para definir los requisitos del sistema desde la perspectiva del usuario final.

En este proyecto se utiliza el formato estándar: "Como [rol], quiero [funcionalidad], para [beneficio]".

Cada historia se registra como un ticket en la herramienta de gestión del equipo, asignada una prioridad y vinculada a un sprint.

ID	Historia de usuario	Prioridad
HU-01	Como usuario, quiero seleccionar ingredientes disponibles en mi casa, para buscar recetas que pueda preparar.	Alta
HU-02	Como usuario, quiero ver los resultados de búsqueda traducidos al español, para entender las recetas sin dificultad.	Alta
HU-03	Como usuario, quiero ver el detalle completo de una receta, para conocer ingredientes, cantidades e instrucciones paso a paso.	Alta
HU-04	Como usuario, quiero ver un indicador de coincidencia de ingredientes, para priorizar las recetas más viables.	Media
HU-05	Como usuario, quiero ver etiquetas de dieta en cada receta, para filtrar opciones según mis restricciones alimentarias.	Media

6. Estrategia de Revisiones y Aprobaciones

La revisión es obligatoria: al menos un miembro del equipo debe aprobar los cambios antes de que se permita la fusión. Durante la revisión se verifican aspectos como la coherencia del código con los estándares del proyecto, la existencia de pruebas para las nuevas funcionalidades, y la actualización de la documentación si es necesario.

- ☐ El código cumple con los estándares de estilo definidos por ESLint y Prettier.
- ☐ Las pruebas unitarias han sido ejecutadas y han dado resultado exitoso.
- ☐ La documentación relevante ha sido actualizada.
- ☐ El pipeline de CI ha finalizado sin errores.
- ☐ Ejecución correcta de pipeline CD.
- ☐ Los cambios no introducen dependencias no necesarias.

2025-b-sw-construccion-gr2 / chicalza-2025-b-amct-sw-gr2

Q

type to search

<> Code

Issues

Pull requests 1

Agents

Actions

Projects 1

Wiki

Security

Insights

Settings

Agregar proyecto inicial y configuración del workflow CI #3

Edit

<> Code

~ Merged

andrea-m11 merged 7 commits into develop from feature/add-ci-workflow now

Conversation 4

Commits 7

Checks 10

Files changed 87

+18,107 -2,686

andrea-m11 commented 33 minutos ago • edited

Member

Resumen

Subida de todos los archivos del proyecto inicial Umami-Recetas.

Se agrega el pipeline de CI/CD con GitHub Actions junto con la configuración de ESLint, Prettier y pruebas unitarias con Jest. El objetivo es automatizar la validación de calidad del código en cada push o PR.

Cambios

- `.github/workflows/ci.yml` — Pipeline de CI que ejecuta lint, format check, tests y build en paralelo, con un status check final como gate de merge.
- `eslint.config.js` / `.prettierrc` — Configuración de linting y formato de código.
- `src/_tests_/` — Tres archivos de pruebas unitarias: `ingredienteservice`, `spoonacularservice` y `traductorservice`, con reporte de cobertura.
- `package.json` — Scripts nuevos: `lint`, `format`, `format:check`, `test`, `build`.

Checklist de revisión

- ☒ El pipeline pasa en verde (lint, format, tests, build)
- ☒ Lint pasa sin errores (ESLint)
- ☒ Format check pasa sin errores (Prettier)
- ☒ Tests pasan y cubren los servicios principales (jest)
- ☒ El build se genera correctamente
- ☒ La cobertura se sube correctamente a Codecov y es mayor al 80%

Reviewers

Suggestions

Copilot

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Unsubscribe

You're receiving notifications because you modified the

Request

Merged

Agregar proyecto inicial y configuración del workflow CI #3

andrea-m11 merged 7 commits into `develop` from `feature/add-ci-wor...` now

1 participant

Mention @copilot in a comment to make changes to this pull request.

Lock conversation

andrea-m11 added 7 commits 2 hours ago

Adición de CI workflow y config linters/tests

5ac2b0d

Agregar pruebas unitarias y reporte de cobertura

ee19c87

Correcciones Services

1a05299

chore: configurar reportes de cobertura de Jest

470b105

Configurar reportes de cobertura de Jest

c04d99e

Documentación en README.md

63040d0

Documentación Final en README.md

8a51c8f

andrea-m11 commented 3 minutes ago

View reviewed changes

`.github/workflows/ci.yml`

22

33

44

55

on:

push:

branches: [main, develop, feature/**]

andrea-m11 10 minutes ago

Member

Author

...

¿Las ramas feature/** están bien definidas en el trigger? Verificar que el patrón captura subramas correctamente.

andrea-m11 2 minutes ago

Member

Author

...

Verificación aprobada, el patrón captura subramas correctamente.

Reply...

Merged

Agregar proyecto inicial y configuración del workflow CI #3

andrea-m11 merged 7 commits into `develop` from `feature/add-ci-wor...` 1 minute ago

`Examen_002/UnamiRecetas/eslint.config.js`

andrea-m11 8 minutes ago

Member

Author

...

Verificar que las reglas configuradas aplican a todos los archivos de src/ sin excepciones.

andrea-m11 2 minutes ago

Member

Author

...

Verificación aprobada, las reglas aplican correctamente a todo el proyecto.

Reply...

Unresolve conversation

andrea-m11 marked this conversation as resolved.

`Examen_002/UnamiRecetas/src/__tests__/spoonacularService.test.js`

andrea-m11 6 minutes ago

Member

Author

...

Tests cubren tanto el caso exitoso como el de error de la API

Reply...

Unresolve conversation

andrea-m11 marked this conversation as resolved.

`Examen_002/UnamiRecetas/src/__tests__/traductorService.test.js`

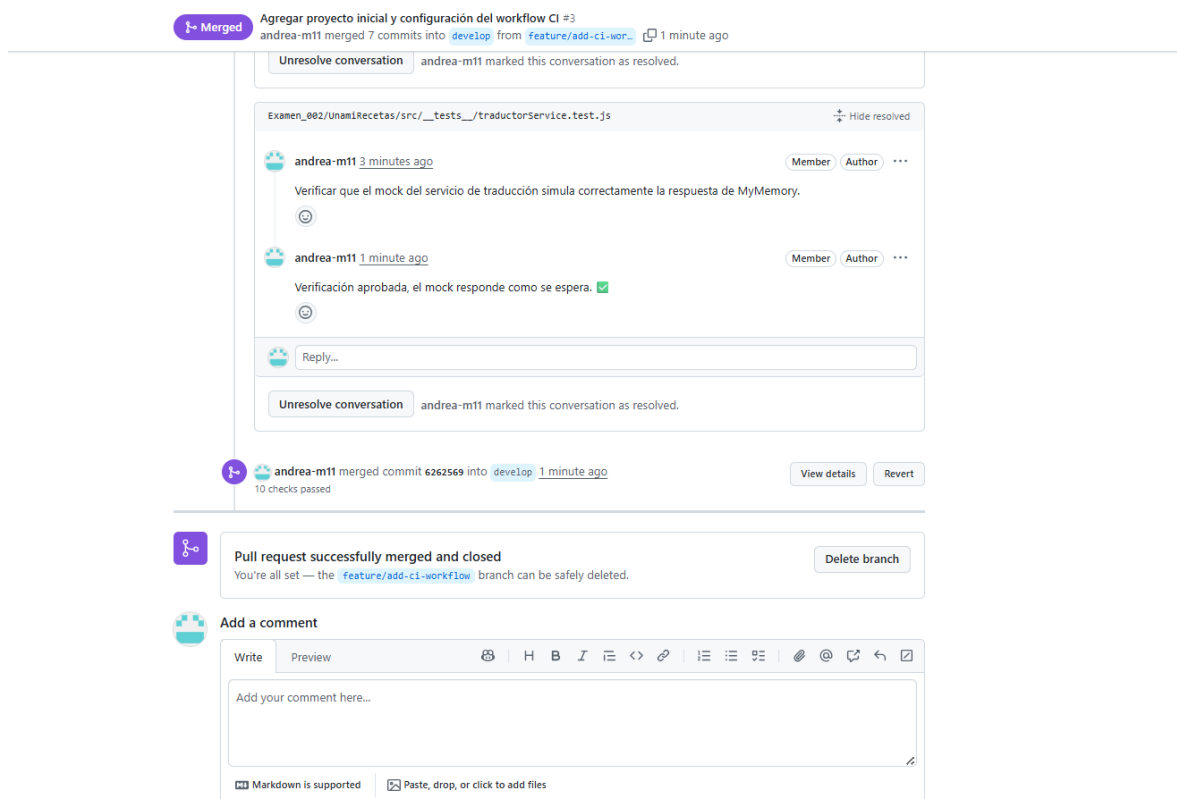
andrea-m11 3 minutes ago

Member

Author

...

Verificar que el mock del servicio de traducción simula correctamente la respuesta de MyMemory.



Este proceso de revisión, combinado con la ejecución automática del pipeline de CI, asegura que solo el código que cumple con los estándares de calidad del proyecto llegue a la rama de desarrollo principal.

7. Herramientas y Conexiones

El proyecto utiliza un conjunto de herramientas integradas que facilitan la gestión del código, la automatización de validaciones y la comunicación entre los miembros del equipo. La conexión entre estas herramientas permite generar un flujo de trabajo eficiente y trazable.

Categoría	Herramienta	Uso en el proyecto
Repositorio de código	GitHub	Almacenamiento del código fuente y gestión de ramas.
CI/CD	GitHub Actions	Automatización del pipeline de integración continua.
Linting	ESLint	Análisis estático del código JavaScript.
Formato de código	Prettier	Estandarización del estilo de escritura del código.
Testing	Jest	Ejecución de pruebas unitarias y medición de cobertura.
Cobertura	Codecov	Reportes visuales de cobertura de código.

GitHub sirve como punto central de integración: los Pull Requests permiten revisar y aprobar cambios antes de fusionarlos, mientras que GitHub Actions ejecuta automáticamente el pipeline

de CI en cada interacción con el repositorio. Los reportes generados por Jest se envían a Codecov, proporcionando una vista consolidada de la calidad del código a lo largo del tiempo.

8. Conclusiones

El proyecto Umami ha sido desarrollado siguiendo prácticas modernas de ingeniería de software que garantizan calidad, trazabilidad y sostenibilidad en largo plazo. La implementación de un pipeline de CI mediante GitHub Actions permitió detectar errores de forma temprana y automática, reduciendo el riesgo de que código defectuoso llegue a las ramas principales del repositorio.

La adopción de la estrategia GitFlow facilitó la organización del trabajo en ramas dedicadas, permitiendo que cada funcionalidad se desarrolle, revise y fusione de forma independiente y ordenada. El uso obligatorio de Pull Requests, junto con la lista de verificación de revisión, aseguró que cada cambio fue evaluado por al menos un miembro del equipo antes de ser integrado.

Las herramientas de análisis estático como ESLint y Prettier contribuyeron a mantener un código homogéneo y libre de errores comunes, mientras que Jest permitió medir y supervisar la cobertura de pruebas a lo largo de todo el proceso de desarrollo. En conjunto, estas estrategias y herramientas constituyeron un marco sólido para la gestión profesional de la construcción y evolución del software Umami.